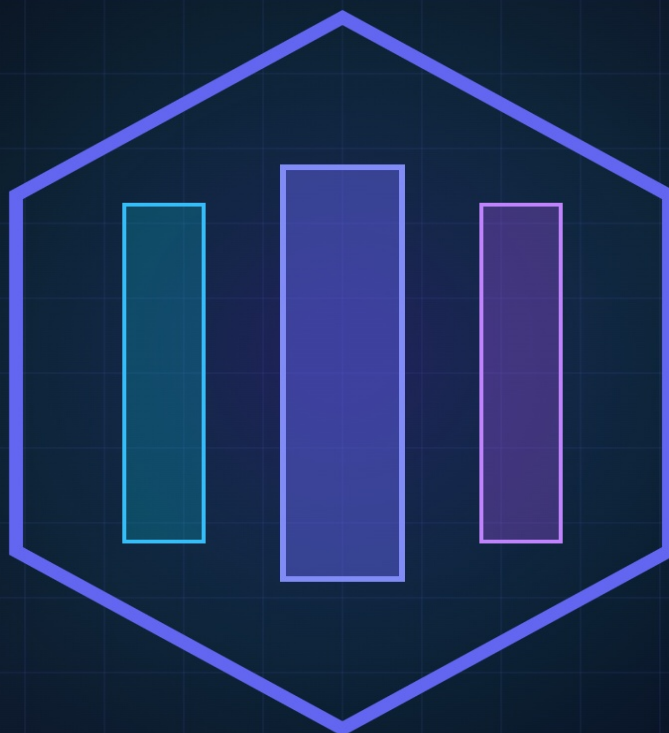


GUÍA DEFINITIVA DE ARQUITECTURA DE SOFTWARE

CONSTRUYENDO SOFTWARE ESCALABLE CON

HexaGenPHP



Vertical Slice Architecture, Hexagonal Design
y **Reactividad Moderna sin SPAs**

HTMX • AES-256-GCM • CQRS • DOMAIN EVENTS • MULTI-TENANCY

POR

Raul Alejandro Baez Camarillo

Tabla de contenido

Introduction	1.1
Introducción	1.2

PARTE I: FUNDAMENTOS Y FILOSOFÍA ARQUITECTÓNICA

Capítulo 1: La Crisis del Monolito Horizontal y la Promesa de VSA	2.1
Capítulo 2: Arquitectura Hexagonal Pragmática en PHP 8.3+	2.2
Capítulo 3: Anatomía del Framework HexaGenPHP	2.3

PARTE II: DESARROLLO CORE Y VERTICAL SLICES

Capítulo 4: Instalación, Tooling y el CLI Generador	3.1
Capítulo 5: Modelado del Dominio: Entities, Value Objects e Inmutabilidad	3.2
Capítulo 6: El Ciclo de Vida de un Slice: DTOs, Validadores y Handlers	3.3
Capítulo 7: Persistencia Desacoplada, Repositorios y Transaccionalidad	3.4

PARTE III: REACTIVIDAD, SEGURIDAD Y EVENTOS AVANZADOS

Capítulo 8: Live Slices y UI Reactiva con HTMX	4.1
Capítulo 9: Criptografía en el Cliente: Blindaje AES-256-GCM del Estado	4.2
Capítulo 10: Arquitectura Guiada por Eventos (Domain Events & Event Bus)	4.3
Capítulo 11: APIs Híbridas: Unificando Web y Mobile en un solo Slice	4.4

PARTE IV: PRODUCCIÓN, ESCALA Y PROYECTO ENTERPRISE

Capítulo 12: Multi-Tenancy y Seguridad Avanzada (RBAC & Permissions)	5.1
Capítulo 13: Proyecto Completo de Extremo a Extremo (SaaS Transaccional)	5.2
Capítulo 14: Testing Automatizado, Dockerización, CI/CD y Rendimiento	5.3

Durante años, la narrativa predominante en la industria del software insistió en que para construir aplicaciones interactivas, rápidas y escalables era obligatorio fragmentar el desarrollo en dos mundos completamente aislados: una API compleja en el backend y una robusta aplicación de página única (*Single Page Application* o SPA) en el frontend construida con React, Vue o Angular.

Esta separación trajo consigo una sobrecarga técnica masiva:

- **Duplicación de modelos y validaciones:** Definir esquemas de datos tanto en PHP como en TypeScript.
- **Fricción operativa desmedida:** Mayor costo de mantenimiento para equipos ágiles y desarrolladores individuales.
- **Deuda técnica acelerada:** Dispersión de reglas de negocio a través de capas horizontales genéricas (*MVC clásico*).

HexaGenPHP nace como una respuesta pragmática a este problema.

¿Qué es este libro?

Este texto no es un compendio teórico abstracto ni una simple transcripción de documentación técnica. Es una **guía práctica de ingeniería de software** diseñada para enseñarte a construir aplicaciones web listas para producción, aplicando dos de los patrones más sólidos de la arquitectura moderna:

1. **Vertical Slice Architecture (VSA):** Organización de software alrededor de características y casos de uso de negocio reales, no de roles técnicos horizontales.
2. **Arquitectura Hexagonal (Puertos y Adaptadores):** Protección estricta del núcleo de negocio frente a la infraestructura, bases de datos o servicios externos.

A lo largo de estas páginas descubrirás cómo **HexaGenPHP** combina estos principios con un motor reactivo nativo de **Live Slices**: interfaces dinámicas y fluidas impulsadas por HTMX, con estado cifrado en cliente mediante **AES-256-GCM** y enrutamiento automático sin configuración manual de endpoints.

¿A quién está dirigido?

- **Desarrolladores PHP (Mid a Senior)** que buscan salir del ciclo de *Fat Models* y controladores sobrecargados en frameworks tradicionales.
 - **Arquitectos de Software y Líderes Técnicos** interesados en entregar productos escalables y mantenibles en tiempos reducidos sin acumular deuda técnica.
 - **Desarrolladores Full-Stack** que desean crear experiencias de usuario reactivas sin la sobrecarga de mantener dos bases de código separadas.
-

Requisitos Previos

Para aprovechar al máximo los ejemplos prácticos de esta guía:

Introduction

- Familiaridad básica con PHP moderno (PHP 8.2+).
 - Un entorno de desarrollo con Composer instalado.
 - Extensiones de PHP activas: `openssl` , `pdo` , `mbstring` .
-

Capítulo 1: La Crisis del Monolito Horizontal y la Promesa de VSA

Durante más de dos décadas, el ecosistema PHP ha estado dominado por el patrón MVC (Modelo-Vista-Controlador) tradicional. Frameworks icónicos popularizaron esta estructura organizando el código en capas horizontales estrictas: una carpeta para todos los controladores, otra para todos los modelos y otra para todas las vistas o servicios.

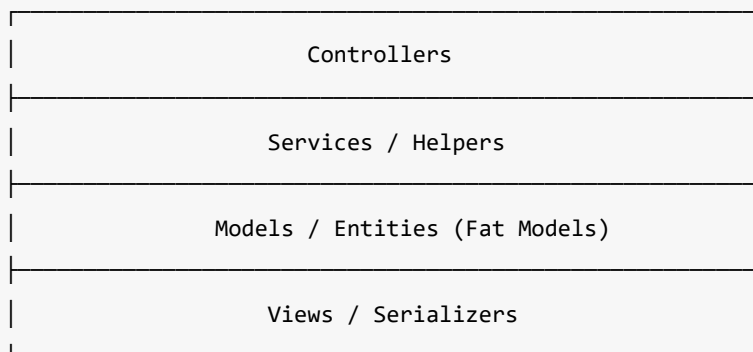
Aunque este enfoque funciona bien para aplicaciones pequeñas o prototipos iniciales, genera fricciones severas a medida que el software escala en complejidad, funcionalidades y tamaño de equipo.

La Trampa del MVC Clásico y las Capas Horizontales

El problema fundamental de la arquitectura horizontal clásica es la **baja cohesión** y el **alto acoplamiento** entre componentes:

- **Fat Controllers y Fat Models:** La lógica de negocio termina dispersa en controladores de miles de líneas o en modelos sobrecargados con métodos de validación, consultas SQL complejas, eventos, mutaciones y transformaciones de datos.
- **Navegación fragmentada:** Para modificar un único flujo de negocio (por ejemplo, *Aprobar una Devolución*), un desarrollador debe abrir y editar simultáneamente entre 5 y 8 archivos en carpetas distantes (`Controllers/` , `Models/` , `Requests/` , `Services/` , `Transformers/` , `Views/`).
- **Efectos secundarios invisibles:** Modificar un método en un modelo o servicio compartido para satisfacer los requisitos de una nueva vista con frecuencia rompe otras vistas o flujos que dependían del comportamiento anterior.

Arquitectura Horizontal Clásica (MVC)



Vertical Slice Architecture (VSA): Cohesión por Caso de Uso

Para resolver la dispersión del MVC horizontal, **Vertical Slice Architecture (VSA)** reorganiza el código alrededor de características de negocio (*Features*) en lugar de roles técnicos.

En lugar de capas horizontales compartidas donde todo se cruza con todo, el sistema se divide en "rebanadas verticales" independientes:

- **Autocontención:** Cada Slice representa un caso de uso específico (ej. `RegisterUser` , `PublishUrgentShift` , `LockEscrowPayment`).
- **Encapsulamiento total:** Cada Slice encapsula su propia entrada (Command/Query DTO), sus reglas de validación, su lógica de negocio y su respuesta visual o JSON.
- **Evolución independiente:** Los Slices no comparten lógica innecesariamente; si dos casos de uso evolucionan de manera diferente, pueden modificarse sin riesgo de regresión en otros módulos del sistema.

Comparativa Visual de Vertical Slice Architecture (VSA)

CreateOrder	CancelOrder	GenerateInvoice
- Request DTO	- Request DTO	- Request DTO
- Validation	- Validation	- Validation
- Domain Logic	- Domain Logic	- Domain Logic
- DB Adapter	- DB Adapter	- DB Adapter
- View / JSON	- View / JSON	- View / JSON

El Principio de Cambio Aislado

El principio fundamental que guía a VSA es: **"Las cosas que cambian juntas deben residir juntas"**.

Cuando un requerimiento del negocio solicita modificar la forma en que se calcula un descuento al crear un pedido:

1. Solo se abre el directorio correspondiente a ese slice (`app/Slices/CreateOrder/`).
2. Se modifican las reglas dentro de ese límite.
3. Se garantiza que ningún otro caso de uso (`CancelOrder` , `CalculateShipping` , etc.) sufrirá efectos secundarios imprevistos.

Esta independencia reduce drásticamente la carga cognitiva del equipo, simplifica la revisión de código (pull requests pequeños y focalizados) y permite escalar tanto la base de código como los equipos de ingeniería de forma predecible.

Capítulo 2: Arquitectura Hexagonal Pragmática en PHP 8.3+

La Arquitectura Hexagonal (también conocida como patrón de *Puertos y Adaptadores*, formulado por Alistair Cockburn) nació con un objetivo primordial: **proteger la lógica de negocio central frente a la volatilidad de las tecnologías e infraestructuras externas**.

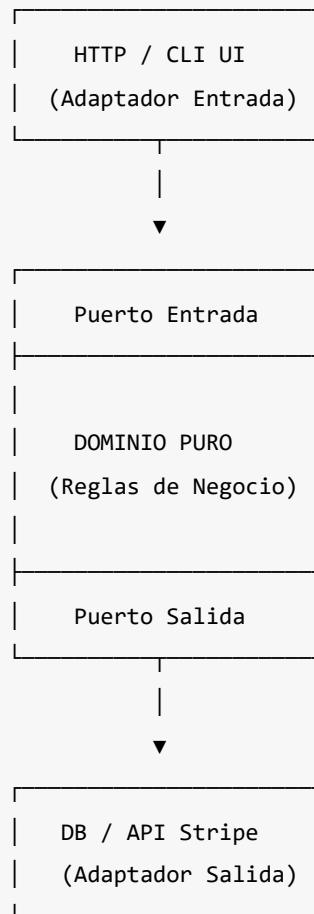
En el desarrollo tradicional, el código del negocio con frecuencia se mezcla de forma inseparable con llamadas a bases de datos específicas (MySQL, PostgreSQL), frameworks web (Laravel, Symfony), librerías de terceros (Stripe, Twilio) o interfaces de usuario. Cuando alguna de estas herramientas externas cambia, la lógica de negocio se ve comprometida.

Anatomía de Puertos y Adaptadores

Para evitar este acoplamiento, la Arquitectura Hexagonal establece límites estrictos mediante tres capas conceptuales:

- **El Dominio Puro (Core):** Contiene exclusivamente las entidades, value objects y reglas de negocio esenciales. No sabe si corre sobre MySQL, PostgreSQL, una petición HTTP, un comando CLI o si se ejecuta dentro de un contenedor Docker.
- **Puertos (Interfaces):** Contratos de código puro que definen qué operaciones necesita el dominio para interactuar con el exterior:
 - *Puertos de Entrada (Driver Ports):* Definen los casos de uso que el exterior puede invocar (ej. `RegisterUserUseCaseInterface`).
 - *Puertos de Salida (Driven Ports):* Definen los servicios externos que el dominio necesita consumir (ej. `UserRepositoryPort` , `PaymentGatewayPort` , `MailerPort`).
- **Adaptadores (Implementaciones):** Código concreto de infraestructura que conecta el exterior con los puertos:
 - *Adaptadores de Entrada:* Controladores HTTP, comandos CLI, listeners de eventos o Webhooks.
 - *Adaptadores de Salida:* Repositorios con PDO/Doctrine/Eloquent, clientes HTTP para Stripe o adaptadores SMTP para correo.

Diagrama de Flujo Hexagonal



Pragmatismo en PHP 8.3+

Históricamente, implementar Arquitectura Hexagonal en PHP requería escribir cientos de líneas de código repetitivo (interfaces vacías, fábricas complejas, clases intermedias y mapeos infinitos).

Con las características modernas de **PHP 8.2 y 8.3+**, esta complejidad se reduce al mínimo:

1. Clases Inmutables y Readonly

Permiten modelar DTOs y Value Objects libres de efectos secundarios con sintaxis concisa:


```

namespace App\Shared\Domain\ValueObjects;

use InvalidArgumentException;

readonly class EmailAddress
{
    public function __construct(public string $value)
    {
        if (!filter_var($this->value, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException("El email '{$this->value}' no tiene un formato
        }
    }
}

```

2. Constructor Property Promotion y Tipado Estricto

Eliminan los setters y getters burocráticos manteniendo la seguridad de tipos:

```

namespace App\Slices\ShiftManagement\Ports;

use App\Slices\ShiftManagement\Entities\Shift;

interface ShiftRepositoryPort
{
    public function findById(string $id): ?Shift;
    public function save(Shift $shift): void;
    public function findAvailableShifts(int $radiusInKm): array;
}

```

La Fusión: Hexagonal dentro de Vertical Slices

HexaGenPHP combina lo mejor de ambos mundos:

1. **La frontera limpia** de la Arquitectura Hexagonal para aislar bases de datos y servicios externos.
2. **La velocidad y enfoque de negocio** de Vertical Slice Architecture para organizar el código por casos de uso.

De este modo, no tienes que crear una estructura hexagonal gigantesca a nivel global; cada slice aplica principios hexagonales en su propio contexto de forma ligera y mantenible.

Capítulo 3: Anatomía del Framework HexaGenPHP

HexaGenPHP fue diseñado bajo una premisa central: **la arquitectura no debe ser un obstáculo burocrático que ralentice la entrega de valor, sino un acelerador que prevenga la degradación del código con el paso del tiempo.**

A diferencia de los frameworks convencionales que imponen estructuras rígidas y monolíticas en capas horizontales, HexaGenPHP adopta un enfoque modular donde cada funcionalidad del sistema vive como una unidad autocontenida y autosuficiente.

Los 4 Pilares Filosóficos del Framework

- **Cohesión Máxima por Caso de Uso:** El código que cambia junto debe vivir junto. En lugar de fragmentar un flujo de negocio en diez directorios técnicos distantes, cada caso de uso reside dentro de su propio slice.
 - **Dominio Agnóstico y Protegido:** Las reglas de negocio esenciales nunca deben depender de bibliotecas de terceros, controladores HTTP o motores de base de datos específicos.
 - **Reactividad Ligera del Lado del Servidor:** Eliminar la sobrecarga operativa y cognitiva de mantener aplicaciones de página única (SPA) separadas en React, Vue o Angular para interactividades comunes de interfaz de usuario.
 - **Seguridad y Cero Manipulación de Estado:** Proveer mecanismos criptográficos nativos para que la reactividad y la comunicación entre cliente y servidor no comprometan la integridad de los datos en ningún momento.
-

La Estructura de Directorios de HexaGenPHP

Una aplicación típica construida con HexaGenPHP organiza sus componentes de la siguiente manera:

```

mi-proyecto/
├─ app/
│   └─ Slices/                # El corazón del sistema: Vertical Slices
│       └─ UserRegistration/
│           └─ RegisterUserCommand.php
│           └─ RegisterUserHandler.php
│           └─ RegisterUserValidator.php
│           └─ view.blade.php / view.php
│       └─ ProcessPayment/
│           └─ ProcessPaymentCommand.php
│           └─ ProcessPaymentHandler.php
│           └─ PaymentLiveSlice.php
│   └─ Shared/                # Componentes y contratos comunes
│       └─ Domain/            # Value Objects e Interfaces base
│       └─ Infrastructure/     # Adaptadores de base de datos, email, etc.
├─ config/                    # Archivos de configuración general
├─ public/                    # Punto de entrada HTTP (index.php) y assets
├─ tests/                     # Pruebas unitarias y de integración
├─ .env                       # Variables de entorno
├─ composer.json
└─ hexagen                    # CLI de generación de código y utilidades

```

HexaGenPHP frente a Otros Enfoques

La siguiente tabla resume cómo se posiciona HexaGenPHP frente a las dos arquitecturas más comunes en la industria actual:

Característica	MVC Tradicional	Microservicios	HexaGenPHP (VSA)
Organización	Capas técnicas horizontales	Servicios distribuidos en red	Slices verticales independientes
Complejidad inicial	Baja	Muy alta	Baja / Moderada
Mantenibilidad	Degrada rápidamente con el tiempo	Alta (a costa de complejidad de red)	Alta y consistente
Reactividad	Requiere SPA separada o jQuery	APIs REST + SPA	Live Slices con estado cifrado
Tiempo de entrega	Rápido al inicio, lento después	Lento por configuración de infraestructura	Rápido y constante

El Rol de los Slices Compartidos (`Shared/`)

Aunque el objetivo de VSA es que los slices sean independientes, ciertas abstracciones son naturalmente transversales a todo el dominio. HexaGenPHP resuelve esto mediante el directorio `app/Shared/` :

- **`Shared/Domain/`** : Value Objects fundamentales (como `EmailAddress` , `Money` , `Uuid`), entidades transversales y contratos base de eventos.
- **`Shared/Infrastructure/`** : Conexiones a bases de datos (PDO), adaptadores de caché (Redis), emisores de correo y clientes de red.

De este modo, se evita la duplicación excesiva de código de bajo nivel mientras se mantiene la pureza e independencia en las capas superiores de cada slice.

Capítulo 4: Instalación, Tooling y el CLI Generador

Para comenzar a construir aplicaciones con HexaGenPHP, es necesario preparar un entorno de desarrollo PHP moderno y familiarizarse con el generador de código por línea de comandos que acelera la creación de componentes arquitectónicos.

Requisitos Previos del Sistema

Antes de iniciar, verifica que tu entorno cumpla con los siguientes requisitos:

- **PHP 8.2 o superior** (se recomienda **PHP 8.3+** para aprovechar las últimas optimizaciones de tipado estricto, constantes en traits y `readonly classes`).
- **Extensiones de PHP requeridas:**
 - `openssl` (indispensable para el cifrado autenticado AES-256-GCM de los Live Slices).
 - `mbstring` , `pdo` , `tokenizer` , `curl` .
- **Composer** (gestor de dependencias de PHP).
- **Servidor Web:** Nginx, Apache o el servidor integrado de PHP para desarrollo local.

Creación de un Nuevo Proyecto

Para inicializar un proyecto con HexaGenPHP mediante Composer, ejecuta en tu terminal:

```
composer create-project rbaezc/hexagenphp mi-aplicacion
```

Una vez completada la descarga, accede al directorio del proyecto:

```
cd mi-aplicacion
```

Configuración de Variables de Entorno

Copia el archivo de ejemplo para generar tu archivo de configuración local:

```
cp .env.example .env
```

Abre el archivo `.env` y configura los parámetros base:

```
APP_NAME="HexaGen App"
APP_ENV=local
APP_KEY=base64:clave_criptografica_generada_aqui
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=hexagen_db
DB_USERNAME=root
DB_PASSWORD=
```

Nota de Seguridad: La directiva `APP_KEY` se utiliza directamente como clave simétrica para firmar y cifrar el estado de los componentes interactivos con AES-256-GCM. Genera siempre una clave segura antes de iniciar el servidor en cualquier entorno.

El Asistente CLI de HexaGen

El ejecutable `hexagen` en la raíz del proyecto es el asistente oficial de scaffolding. Permite crear, validar y gestionar componentes en segundos sin escribir código repetitivo manualmente.

Para consultar la lista completa de comandos disponibles:

```
php hexagen list
```

1. Creación de un Vertical Slice Estándar

Para generar un nuevo caso de uso transaccional (por ejemplo, el registro de un nuevo usuario):

```
php hexagen make:slice UserRegistration
```

Este comando genera de forma automática la estructura necesaria dentro del directorio

```
app/Slices/UserRegistration/ :
```

- `UserRegistrationCommand.php` : El DTO inmutable con las propiedades del caso de uso.
- `UserRegistrationValidator.php` : Las reglas de validación del payload de entrada.
- `UserRegistrationHandler.php` : La clase controladora del caso de uso con inyección de dependencias lista.
- `index.blade.php` : La plantilla de vista asociada.

2. Creación de un Live Slice (Reactivo)

Si el caso de uso requiere interactividad en tiempo real (como un buscador predictivo, un carrito de compras interactivo o un panel de métricas en vivo):

```
php hexagen make:live OrderStatusTracker
```

Esto generará la clase que extiende de `LiveComponent` junto con su plantilla interactiva vinculada a HTMX, lista para manipular estados reactivos.

Inicialización del Servidor Local

Puedes levantar el entorno de pruebas local ejecutando el servidor integrado de PHP:

```
php -S localhost:8000 -t public
```

Al abrir `http://localhost:8000` en tu navegador, verás la pantalla de bienvenida de HexaGenPHP confirmando que el entorno está listo para comenzar a codificar.

Capítulo 5: Modelado del Dominio: Entities, Value Objects e Inmutabilidad

Uno de los errores más comunes en el desarrollo con frameworks tradicionales es el uso de **Modelos Anémicos**: clases que actúan como meros contenedores de propiedades con getters y setters, mientras que la verdadera lógica de negocio queda dispersa en controladores o servicios gigantescos.

En HexaGenPHP, el modelado del dominio es el corazón del sistema. Cada regla de negocio se protege mediante dos bloques fundamentales: **Objetos de Valor (Value Objects)** y **Entidades Ricas (Rich Entities)**.

Value Objects: Inmutabilidad y Validación Continua

Un **Value Object (VO)** es un objeto que no se define por un identificador único, sino por el valor de sus atributos. Dos Value Objects con los mismos atributos son exactamente iguales e intercambiables.

Características Clave de un Value Object

1. **Inmutabilidad:** Una vez instanciado, sus propiedades nunca deben mutar. Si se realiza una operación, se devuelve una nueva instancia.
2. **Auto-validación:** No puede existir un Value Object en un estado inválido. La validación ocurre en el constructor.

Ejemplo Práctico: Money y EmailAddress

```
namespace App\Shared\Domain\ValueObjects;

use InvalidArgumentException;

readonly class Money
{
    public function __construct(
        public int $amountInCents,
        public string $currency = 'MXN'
    ) {
        if ($this->amountInCents < 0) {
            throw new InvalidArgumentException("El monto no puede ser negativo.");
        }
    }

    public static function fromFloat(float $amount, string $currency = 'MXN'): self
    {
        return new self((int) round($amount * 100), $currency);
    }

    public function add(Money $other): self
    {
        if ($this->currency !== $other->currency) {
            throw new InvalidArgumentException("No se pueden sumar montos de diferentes moneda");
        }

        return new self($this->amountInCents + $other->amountInCents, $this->currency);
    }

    public function toFormattedString(): string
    {
        return sprintf("%.2f %s", $this->amountInCents / 100, $this->currency);
    }
}
```

```
namespace App\Shared\Domain\ValueObjects;

use InvalidArgumentException;

readonly class EmailAddress
{
    public function __construct(public string $value)
    {
        $sanitized = trim(strtolower($this->value));
        if (!filter_var($sanitized, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException("El formato del correo electrónico '{$this->val
        }
    }

    public function equals(EmailAddress $other): bool
    {
        return $this->value === $other->value;
    }
}
```

Estados del Dominio con Backed Enums

En PHP 8.1+, los **Backed Enums** proporcionan una forma segura y fuertemente tipada de representar estados y transiciones en el dominio:

```
namespace App\Slices\ShiftManagement\Enums;

enum ShiftStatus: string
{
    case OPEN = 'OPEN';
    case CLAIMED = 'CLAIMED';
    case IN_PROGRESS = 'IN_PROGRESS';
    case COMPLETED = 'COMPLETED';
    case CANCELLED = 'CANCELLED';

    public function isAvailable(): bool
    {
        return $this === self::OPEN;
    }
}
```

Entidades Ricas vs Modelos Anémicos

A diferencia de un Value Object, una **Entidad** tiene una identidad única que perdura a través del tiempo, incluso si todas sus demás propiedades cambian.

Una **Entidad Rica** protege sus invariantes de negocio internamente. Nadie desde el exterior debe poder cambiar su estado sin pasar por métodos explícitos del dominio:

```

namespace App\Slices\ShiftManagement\Entities;

use App\Shared\Domain\ValueObjects\Money;
use App\Slices\ShiftManagement\Enums\ShiftStatus;
use DomainException;

final class Shift
{
    private function __construct(
        private readonly string $id,
        private string $title,
        private Money $hourlyRate,
        private ShiftStatus $status,
        private ?string $claimedByWorkerId = null
    ) {}

    public static function create(string $id, string $title, Money $hourlyRate): self
    {
        return new self(
            id: $id,
            title: $title,
            hourlyRate: $hourlyRate,
            status: ShiftStatus::OPEN
        );
    }

    public function claimBy(string $workerId): void
    {
        if (!$this->status->isAvailable()) {
            throw new DomainException("El turno no está disponible para ser reclamado.");
        }

        $this->status = ShiftStatus::CLAIMED;
        $this->claimedByWorkerId = $workerId;
    }

    public function toState(): array
    {
        return [
            'id' => $this->id,
            'title' => $this->title,
            'hourly_rate' => $this->hourlyRate->amountInCents,
            'currency' => $this->hourlyRate->currency,
            'status' => $this->status->value,
        ];
    }
}

```

```
        'claimed_by_worker_id' => $this->claimedByWorkerId
    ];
}

public static function fromState(array $state): self
{
    return new self(
        id: $state['id'],
        title: $state['title'],
        hourlyRate: new Money((int) $state['hourly_rate'], $state['currency'] ?? 'MXN'),
        status: ShiftStatus::from($state['status']),
        claimedByWorkerId: $state['claimed_by_worker_id'] ?? null
    );
}
```

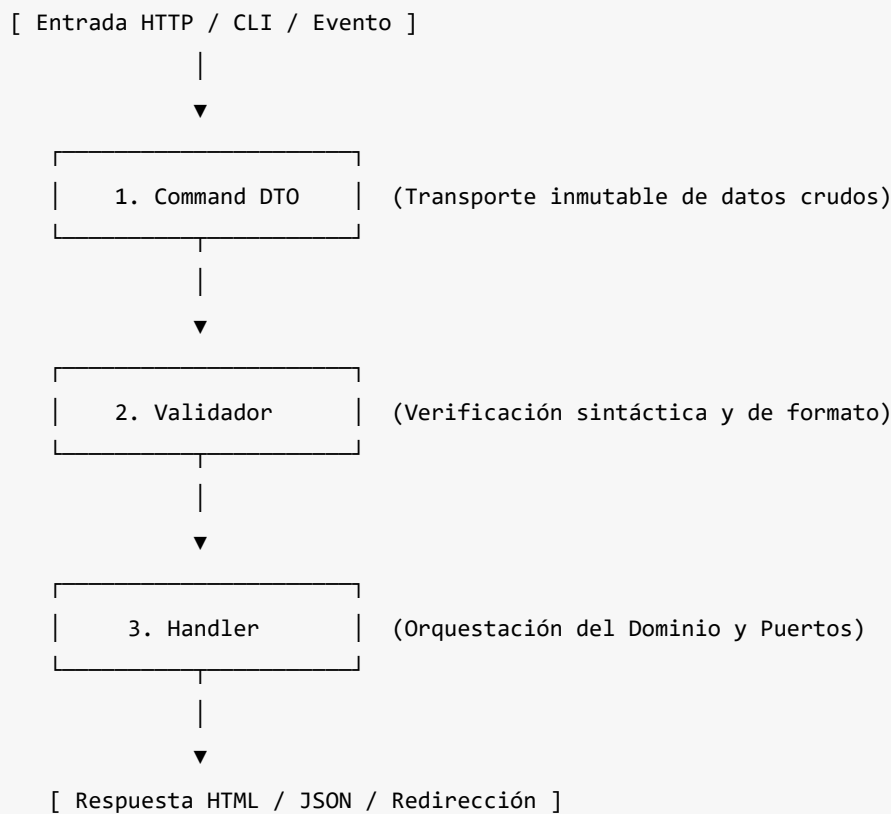
Al utilizar `fromState()` y `toState()`, la entidad no depende de ningún ORM como Doctrine o Eloquent, permitiendo que el dominio permanezca 100% puro y testeable sin bases de datos.

Capítulo 6: El Ciclo de Vida de un Slice: DTOs, Validadores y Handlers

En una arquitectura por cortes verticales (VSA), cada funcionalidad del sistema se ejecuta a través de un flujo predecible, seguro y autocontenido. No hay controladores gigantes ni dependencias ocultas entre casos de uso.

La interacción estándar se estructura alrededor de tres componentes principales: el **Command/Query DTO**, el **Validador** y el **Handler**.

El Flujo de Ejecución de un Slice



1. El Command DTO (Intención del Usuario)

El **Command DTO** representa la intención explícita de ejecutar una acción en el sistema. Debe ser completamente inmutable y transportar únicamente datos primitivos o Value Objects:

```
// app/Slices/UserRegistration/RegisterUserCommand.php
namespace App\Slices\UserRegistration;

final readonly class RegisterUserCommand
{
    public function __construct(
        public string $email,
        public string $password,
        public string $fullName
    ) {}
}
```

2. El Validador del Slice

El validador asegura que la información recibida cumpla con todas las condiciones sintácticas y de estructura antes de invocar la lógica central:

```
// app/Slices/UserRegistration/RegisterUserValidator.php
namespace App\Slices\UserRegistration;

use InvalidArgumentException;

final class RegisterUserValidator
{
    public function validate(RegisterUserCommand $command): void
    {
        $errors = [];

        if (empty(trim($command->fullName))) {
            $errors['fullName'] = 'El nombre completo es obligatorio.';
        }

        if (!filter_var($command->email, FILTER_VALIDATE_EMAIL)) {
            $errors['email'] = 'El formato del correo electrónico es inválido.';
        }

        if (strlen($command->password) < 8) {
            $errors['password'] = 'La contraseña debe contener al menos 8 caracteres.';
        }

        if (!empty($errors)) {
            throw new InvalidArgumentException(json_encode($errors));
        }
    }
}
```

3. El Handler (Orquestador del Caso de Uso)

El **Handler** es el único responsable de coordinar la ejecución del caso de uso. Recibe las dependencias requeridas mediante inyección en el constructor (a través de puertos) y procesa la transacción:


```
// app/Slices/UserRegistration/RegisterUserHandler.php
namespace App\Slices\UserRegistration;

use App\Slices\UserRegistration\Ports\UserRepositoryPort;
use App\Slices\UserRegistration\Ports>PasswordHasherPort;

final class RegisterUserHandler
{
    public function __construct(
        private readonly UserRepositoryPort $repository,
        private readonly PasswordHasherPort $hasher,
        private readonly RegisterUserValidator $validator
    ) {}

    public function handle(RegisterUserCommand $command): RegisterUserResponse
    {
        // 1. Validar payload de entrada
        $this->validator->validate($command);

        // 2. Ejecutar transformaciones y llamadas a puertos
        $hashedPassword = $this->hasher->hash($command->password);
        $user = $this->repository->create($command->email, $hashedPassword, $command->fullName);

        // 3. Devolver DTO de respuesta seguro
        return new RegisterUserResponse(
            userId: $user->id,
            email: $user->email,
            registeredAt: (new \DateTimeImmutable())->format(\DateTimeInterface::ATOM)
        );
    }
}
```

El DTO de Respuesta (`RegisterUserResponse.php`)

```
// app/Slices/UserRegistration/RegisterUserResponse.php
namespace App\Slices\UserRegistration;

final readonly class RegisterUserResponse
{
    public function __construct(
        public string $userId,
        public string $email,
        public string $registeredAt
    ) {}
}
```

Beneficios de esta Estructura

- **Testabilidad Inmediata:** El Handler puede probarse en milisegundos inyectando mocks o implementaciones en memoria de los puertos.
- **Reutilización Multicanal:** El mismo Handler y Command pueden ser invocados desde una ruta HTTP web, un comando por consola (`php hexagen ...`), un worker de cola o una API REST sin duplicar una sola línea de lógica.

Capítulo 7: Persistencia Desacoplada, Repositorios y Transaccionalidad

En una arquitectura limpia, la base de datos es un detalle de infraestructura y no el eje central del software. En HexaGenPHP, los slices definen contratos (**Puertos**) para la persistencia de datos, mientras que las implementaciones concretas (**Adaptadores**) se encargan de interactuar con el motor SQL o NoSQL.

1. Definiendo el Puerto (Contrato de Salida)

El puerto es una interfaz pura que reside dentro del slice o módulo del dominio. Define las operaciones necesarias sin exponer conceptos específicos de la base de datos:

```
// app/Slices/ShiftManagement/Ports/ShiftRepositoryPort.php
namespace App\Slices\ShiftManagement\Ports;

use App\Slices\ShiftManagement\Entities\Shift;

interface ShiftRepositoryPort
{
    public function findById(string $id): ?Shift;
    public function save(Shift $shift): void;
    public function findAvailableShifts(int $radiusInKm): array;
}
```

2. Implementando el Adaptador (Infraestructura con PDO)

El adaptador implementa el contrato del puerto utilizando PDO con sentencias preparadas de alto rendimiento:

```
// app/Shared/Infrastructure/Persistence/PdoShiftRepository.php
namespace App\Shared\Infrastructure\Persistence;

use App\Slices\ShiftManagement\Ports\ShiftRepositoryPort;
use App\Slices\ShiftManagement\Entities\Shift;
use PDO;

final class PdoShiftRepository implements ShiftRepositoryPort
{
    public function __construct(private readonly PDO $pdo) {}

    public function findById(string $id): ?Shift
    {
        $stmt = $this->pdo->prepare("SELECT * FROM shifts WHERE id = :id LIMIT 1");
        $stmt->execute(['id' => $id]);
        $row = $stmt->fetch(PDO::FETCH_ASSOC);

        return $row ? Shift::fromState($row) : null;
    }

    public function save(Shift $shift): void
    {
        $data = $shift->toState();
        $stmt = $this->pdo->prepare("
            INSERT INTO shifts (id, title, hourly_rate, currency, status, claimed_by_worker_id)
            VALUES (:id, :title, :hourly_rate, :currency, :status, :claimed_by_worker_id)
            ON DUPLICATE KEY UPDATE
                status = VALUES(status),
                claimed_by_worker_id = VALUES(claimed_by_worker_id)
        ");
        $stmt->execute($data);
    }

    public function findAvailableShifts(int $radiusInKm): array
    {
        $stmt = $this->pdo->query("SELECT * FROM shifts WHERE status = 'OPEN' ORDER BY created");
        return array_map(fn($row) => Shift::fromState($row), $stmt->fetchAll(PDO::FETCH_ASSOC));
    }
}
```

3. Manejo de Transaccionalidad Atómica

Para operaciones críticas que involucren múltiples tablas (como cobrar un pago y asignar un turno), se utiliza un puerto de transacciones:

```
// app/Shared/Domain/Ports/TransactionManagerPort.php
namespace App\Shared\Domain\Ports;

use Closure;

interface TransactionManagerPort
{
    public function transactional(Closure $operation): mixed;
}
```

Adaptador de Transacción PDO:

```
// app/Shared/Infrastructure/Persistence/PdoTransactionManager.php
namespace App\Shared\Infrastructure\Persistence;

use App\Shared\Domain\Ports\TransactionManagerPort;
use PDO;
use Throwable;

final class PdoTransactionManager implements TransactionManagerPort
{
    public function __construct(private readonly PDO $pdo) {}

    public function transactional(\Closure $operation): mixed
    {
        $this->pdo->beginTransaction();
        try {
            $result = $operation();
            $this->pdo->commit();
            return $result;
        } catch (Throwable $e) {
            $this->pdo->rollBack();
            throw $e;
        }
    }
}
```

4. Inyección de Dependencias en el Contenedor

Para enlazar las interfaces con sus implementaciones concretas, registra las dependencias en el contenedor de servicios:

```
// config/container.php
$container->bind(
    \App\Slices\ShiftManagement\Ports\ShiftRepositoryPort::class,
    \App\Shared\Infrastructure\Persistence\PdoShiftRepository::class
);

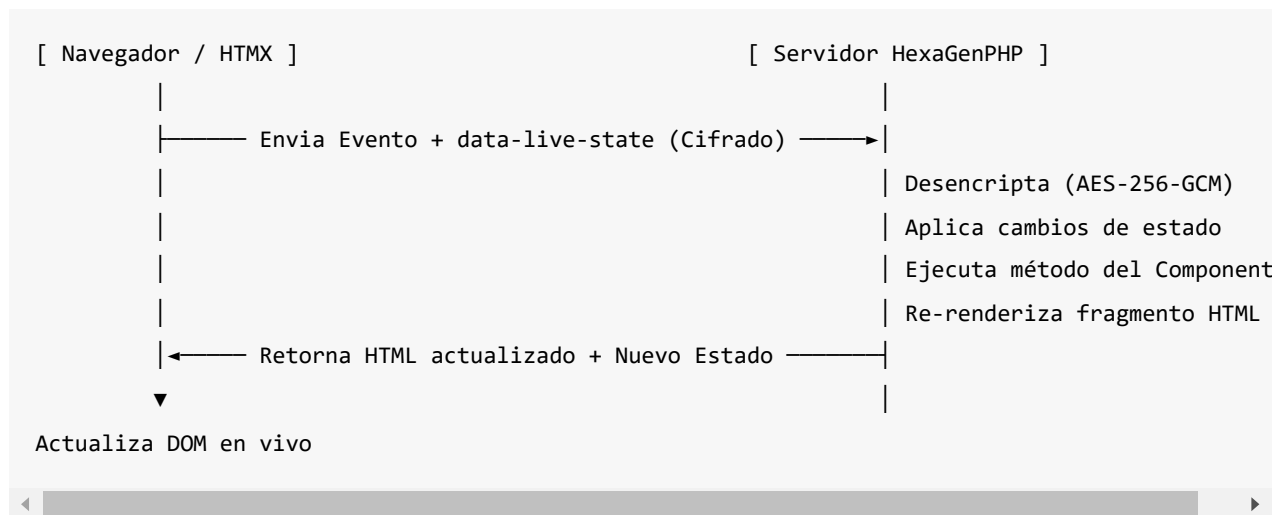
$container->bind(
    \App\Shared\Domain\Ports\TransactionManagerPort::class,
    \App\Shared\Infrastructure\Persistence\PdoTransactionManager::class
);
```

Capítulo 8: Live Slices y UI Reactiva con HTMX

Los **Live Slices** son el mecanismo nativo de HexaGenPHP para ofrecer experiencias interactivas y fluidas del lado del cliente sin la necesidad de construir APIs REST redundantes ni mantener aplicaciones complejas compiladas en React, Vue o Angular.

El Ciclo de Vida de un Live Slice

El flujo de reactividad se basa en el intercambio eficiente de fragmentos HTML mediante **HTMX**, manteniendo sincronizado el estado del componente entre el cliente y el servidor:



El Endpoint Automático `/live/{Component}/{method}`

HexaGenPHP incluye un enrutador inteligente para interactuar con los componentes en tiempo real:

- **Cero rutas manuales:** No necesitas registrar endpoints HTTP individuales para cada botón interactivo, toggle o buscador.
- **Enrutamiento por convención:** Cualquier petición dirigida a `/live/{NombreComponente}/{nombreMetodo}` es despachada de inmediato al método correspondiente del Live Slice.

Ejemplo Práctico: Contador Reactivo con Live Slice

1. Definición del Componente PHP

```
// app/Slices/Counter/CounterLiveSlice.php
namespace App\Slices\Counter;

use HexaGen\Live\LiveComponent;

final class CounterLiveSlice extends LiveComponent
{
    public int $count = 0;

    public function increment(): void
    {
        $this->count++;
    }

    public function decrement(): void
    {
        if ($this->count > 0) {
            $this->count--;
        }
    }

    public function render(): string
    {
        return $this->view('Counter/view', [
            'count' => $this->count
        ]);
    }
}
```


2. Plantilla de Vista Blade con HTMX

```
<!-- app/Slices/Counter/view.blade.php -->
<div class="p-6 bg-white rounded-xl shadow-md space-y-4 max-w-sm mx-auto text-center"
    data-live-component="CounterLiveSlice"
    data-live-state="{{ $this->getEncryptedState() }}">

    <h2 class="text-xl font-bold text-slate-800">Contador Reactivo</h2>

    <div class="text-4xl font-extrabold text-indigo-600">
        {{ $count }}
    </div>

    <div class="flex justify-center gap-3">
        <button hx-post="/live/CounterLiveSlice/decrement"
            hx-target="closest [data-live-component]"
            hx-swap="outerHTML"
            class="px-4 py-2 bg-slate-200 hover:bg-slate-300 rounded-lg font-semibold text-
                - Disminuir
        </button>

        <button hx-post="/live/CounterLiveSlice/increment"
            hx-target="closest [data-live-component]"
            hx-swap="outerHTML"
            class="px-4 py-2 bg-indigo-600 hover:bg-indigo-700 rounded-lg font-semibold te
                + Aumentar
        </button>
    </div>
</div>
```

Capítulo 9: Criptografía en el Cliente: Blindaje AES-256-GCM del Estado

En cualquier arquitectura reactiva basada en servidor existen dos grandes desafíos al gestionar el estado de los componentes interactivos:

1. **Almacenar el estado en la memoria RAM del servidor:** Requiere sesiones pegajosas (*sticky sessions*), clústeres de Redis dedicados y alto consumo de memoria que colapsa ante picos de tráfico.
2. **Enviar el estado en texto plano al cliente:** Expone datos sensibles y permite que usuarios maliciosos alteren precios, roles o identificadores directamente desde la consola del navegador.

HexaGenPHP resuelve ambos problemas adoptando un modelo sin estado en servidor (*stateless server*) con **cifrado simétrico autenticado AES-256-GCM** en el cliente.

Anatomía del Cifrado AES-256-GCM

AES-256-GCM (*Galois/Counter Mode*) es un algoritmo de cifrado autenticado que proporciona simultáneamente **confidencialidad** (los datos son ilegibles) e **integridad autenticada** (garantiza que los datos no fueron modificados ni alterados en tránsito).

Estructura del Payload Cifrado

Cada vez que un Live Slice renderiza su vista, genera una cadena codificada en Base64 que contiene tres elementos:

Vector de Inicio (IV)	Cuerpo Cifrado (Data)	Tag de Autenticación
(12 bytes)	(Longitud N)	(16 bytes)

Implementación Criptográfica en HexaGenPHP

A continuación se muestra el motor criptográfico interno que utiliza el framework para proteger el atributo `data-live-state` :

```

namespace HexaGen\Live\Security;

use RuntimeException;

final class StateCrypter
{
    private const CIPHER = 'aes-256-gcm';
    private const IV_LENGTH = 12;
    private const TAG_LENGTH = 16;

    public function __construct(private readonly string $key) {}

    public function encrypt(array $state): string
    {
        $serialized = json_encode($state, JSON_THROW_ON_ERROR);
        $iv = random_bytes(self::IV_LENGTH);
        $tag = '';

        $ciphertext = openssl_encrypt(
            $serialized,
            self::CIPHER,
            $this->key,
            OPENSSL_RAW_DATA,
            $iv,
            $tag,
            '',
            self::TAG_LENGTH
        );

        if ($ciphertext === false) {
            throw new RuntimeException("Error al cifrar el estado del componente.");
        }

        // Empaquetar IV + TAG + CIPHERTEXT
        $payload = $iv . $tag . $ciphertext;

        return base64_encode($payload);
    }

    public function decrypt(string $encryptedPayload): array
    {
        $decoded = base64_decode($encryptedPayload, true);
        if ($decoded === false || strlen($decoded) < (self::IV_LENGTH + self::TAG_LENGTH)) {
            throw new RuntimeException("Payload criptográfico corrupto o inválido.");
        }
    }
}

```

```

    }

    $iv = substr($decoded, 0, self::IV_LENGTH);
    $tag = substr($decoded, self::IV_LENGTH, self::TAG_LENGTH);
    $ciphertext = substr($decoded, self::IV_LENGTH + self::TAG_LENGTH);

    $decrypted = openssl_decrypt(
        $ciphertext,
        self::CIPHER,
        $this->key,
        OPENSSL_RAW_DATA,
        $iv,
        $tag
    );

    if ($decrypted === false) {
        throw new RuntimeException("Fallo de autenticación: El estado ha sido alterado o m
    }

    return json_decode($decrypted, true, 512, JSON_THROW_ON_ERROR);
}
}

```

Resistencia Total a Manipulación (*Tamper Resistance*)

Si un atacante modifica un solo carácter del atributo `data-live-state` en el navegador para intentar alterar un valor:

1. `openssl_decrypt` verifica el tag de autenticación (Tag GCM).
2. La firma criptográfica falla de inmediato.
3. La petición es rechazada con un código HTTP `400 Bad Request` antes de ejecutar cualquier método del componente.

Este blindaje permite que tus servidores escalen horizontalmente de forma masiva sin requerir memoria compartida, garantizando una seguridad de grado bancario.

Capítulo 10: Arquitectura Guiada por Eventos (Domain Events & Event Bus)

En aplicaciones de gran escala, un caso de uso rara vez termina con guardar un registro en la base de datos. Con frecuencia, un evento de negocio desencadena múltiples acciones secundarias: enviar una notificación por correo, registrar una auditoría, emitir un Webhook a un socio comercial y actualizar contadores en tiempo real.

Si colocamos todas estas acciones dentro del mismo Handler, violamos el principio de responsabilidad única y creamos un cuello de botella. La **Arquitectura Guiada por Eventos (EDA)** desacopla completamente el flujo principal de los procesos secundarios.

1. Modelado de Eventos del Dominio

Un **Domain Event** es un registro inmutable de algo significativo que ya ocurrió en el negocio. Se nombra en pasado:

```
// app/Shared/Domain/Events/DomainEventInterface.php
namespace App\Shared\Domain\Events;

use DateTimeImmutable;

interface DomainEventInterface
{
    public function occurredOn(): DateTimeImmutable;
}
```

```
// app/Slices/ShiftManagement/Events/ShiftClaimedDomainEvent.php
namespace App\Slices\ShiftManagement\Events;

use App\Shared\Domain\Events\DomainEventInterface;
use DateTimeImmutable;

final readonly class ShiftClaimedDomainEvent implements DomainEventInterface
{
    public function __construct(
        public string $shiftId,
        public string $workerId,
        public DateTimeImmutable $occurredOn = new DateTimeImmutable()
    ) {}

    public function occurredOn(): DateTimeImmutable
    {
        return $this->occurredOn;
    }
}
```

2. El Puerto del Event Bus

El Dominio emite eventos a través de una interfaz abstracta:

```
// app/Shared/Domain/Ports/EventBusPort.php
namespace App\Shared\Domain\Ports;

use App\Shared\Domain\Events\DomainEventInterface;

interface EventBusPort
{
    public function publish(DomainEventInterface ...$events): void;
}
```

3. Implementación del Event Bus Sincrónico / Asíncrono

```
// app/Shared/Infrastructure/Events/InMemoryEventBus.php
namespace App\Shared\Infrastructure\Events;

use App\Shared\Domain\Ports\EventBusPort;
use App\Shared\Domain\Events\DomainEventInterface;

final class InMemoryEventBus implements EventBusPort
{
    private array $listeners = [];

    public function subscribe(string $eventClass, callable $listener): void
    {
        $this->listeners[$eventClass][] = $listener;
    }

    public function publish(DomainEventInterface ...$events): void
    {
        foreach ($events as $event) {
            $eventClass = get_class($event);
            if (isset($this->listeners[$eventClass])) {
                foreach ($this->listeners[$eventClass] as $listener) {
                    $listener($event);
                }
            }
        }
    }
}
```

4. Emisión y Consumo en Slices Separados

Publicación desde el Handler:

```
// Dentro de ClaimShiftHandler.php
$this->repository->save($shift);

// Publicar el evento de dominio
$this->eventBus->publish(new ShiftClaimedDomainEvent(
    shiftId: $shift->id,
    workerId: $command->workerId
));
```

Listener Desacoplado:

```
// app/Slices/Notifications/Listeners/SendShiftClaimedEmailListener.php
namespace App\Slices\Notifications\Listeners;

use App\Slices\ShiftManagement\Events\ShiftClaimedDomainEvent;
use App\Shared\Domain\Ports\MailerPort;

final class SendShiftClaimedEmailListener
{
    public function __construct(private readonly MailerPort $mailer) {}

    public function __invoke(ShiftClaimedDomainEvent $event): void
    {
        $this->mailer->send(
            to: "worker-{$event->workerId}@empresa.com",
            subject: "¡Turno asignado exitosamente!",
            body: "Has reclamado el turno #{ $event->shiftId } con éxito."
        );
    }
}
```


Capítulo 11: APIs Híbridas: Unificando Web y Mobile en un solo Slice

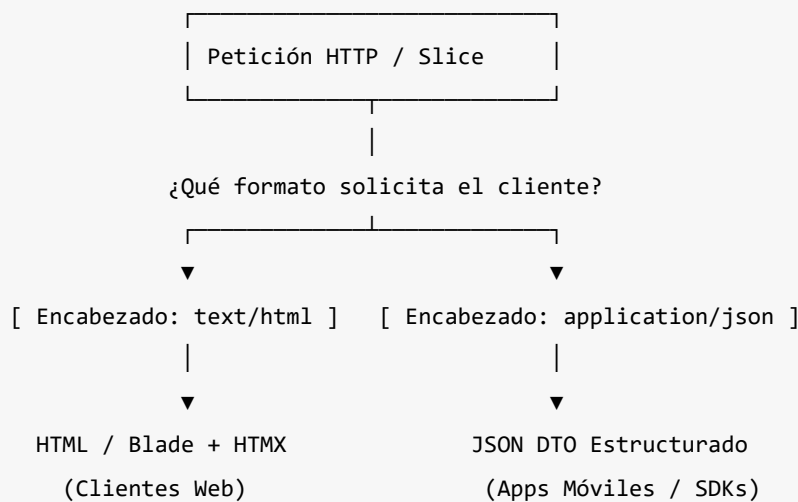
Uno de los mayores causantes de deuda técnica en proyectos modernos es la duplicación de rutas y controladores: uno para servir vistas web y otro idéntico para alimentar aplicaciones móviles (iOS, Android, Flutter o React Native).

HexaGenPHP introduce el concepto de **Slices Híbridos Multicanal**, permitiendo que un único caso de uso sirva de forma transparente tanto a la interfaz web reactiva como a clientes nativos o APIs externas.

Negociación de Contenido Inteligente

En lugar de crear dos endpoints independientes, el Slice analiza los encabezados HTTP de la petición entrante:

- Si la petición proviene de HTMX o un navegador web (`HX-Request: true` o `Accept: text/html`): Retorna el fragmento HTML reactivo con su estado cifrado listo para ser inyectado en el DOM.
- Si la petición proviene de una App Móvil (`Accept: application/json` o token Bearer): Retorna un payload JSON estandarizado con el DTO de respuesta.



Implementación de un Responder Híbrido

```
// app/Shared/Infrastructure/Http/HybridResponder.php
namespace App\Shared\Infrastructure\Http;

final class HybridResponder
{
    public static function respond(
        mixed $data,
        string $viewTemplate,
        int $statusCode = 200
    ): void {
        $acceptHeader = $_SERVER['HTTP_ACCEPT'] ?? '';
        $isJsonRequest = str_contains($acceptHeader, 'application/json');

        http_response_code($statusCode);

        if ($isJsonRequest) {
            header('Content-Type: application/json; charset=utf-8');
            echo json_encode([
                'success' => $statusCode >= 200 && $statusCode < 300,
                'data' => $data
            ], JSON_THROW_ON_ERROR);
            exit;
        }

        // Renderizar vista HTML / Blade
        header('Content-Type: text/html; charset=utf-8');
        echo render_template($viewTemplate, ['data' => $data]);
        exit;
    }
}
```

Consumo desde el Slice

```
// app/Slices/GetShiftDetails/GetShiftDetailsAction.php
namespace App\Slices\GetShiftDetails;

use App\Shared\Infrastructure\Http\HybridResponder;

final class GetShiftDetailsAction
{
    public function __construct(private readonly GetShiftDetailsHandler $handler) {}

    public function __invoke(string $id): void
    {
        $responseDto = $this->handler->handle(new GetShiftDetailsQuery($id));

        HybridResponder::respond(
            data: $responseDto,
            viewTemplate: 'GetShiftDetails/view'
        );
    }
}
```

Ventajas de las APIs Híbridas

- **Cero duplicación:** Las reglas de negocio, autorizaciones y validaciones se escriben una sola vez en el Handler.
- **Mantenimiento simplificado:** Cualquier cambio o corrección de bugs impacta positivamente a la web y a la app móvil de inmediato.

Capítulo 12: Multi-Tenancy y Seguridad Avanzada (RBAC & Permissions)

La mayoría de las aplicaciones modernas de software como servicio (SaaS) deben ser capaces de servir a miles de organizaciones o empresas clientes (llamadas *Tenants* o inquilinos) compartiendo la misma infraestructura, garantizando al mismo tiempo un **aislamiento absoluto de datos** y un sistema granular de control de acceso.

1. El Contexto de Inquilino Inmutable (*TenantContext*)

Para evitar que un desarrollador olvide filtrar consultas por la empresa correspondiente, HexaGenPHP utiliza un objeto de contexto inyectable en el contenedor de servicios:

```
// app/Shared/Domain/Context/TenantContext.php
namespace App\Shared\Domain\Context;

use RuntimeException;

final class TenantContext
{
    private static ?string $currentTenantId = null;

    public static function set(string $tenantId): void
    {
        self::$currentTenantId = $tenantId;
    }

    public static function get(): string
    {
        if (self::$currentTenantId === null) {
            throw new RuntimeException("No se ha establecido un Tenant activo en el contexto d
        }

        return self::$currentTenantId;
    }
}
```

2. Control de Acceso Basado en Roles (RBAC)

Para gestionar los permisos de los usuarios dentro de cada organización, definimos un modelo declarativo de roles y permisos:

```
// app/Shared/Domain/Enums/Permission.php
namespace App\Shared\Domain\Enums;

enum Permission: string
{
    case VIEW_SHIFTS = 'shifts:view';
    case CREATE_SHIFTS = 'shifts:create';
    case CLAIM_SHIFTS = 'shifts:claim';
    case MANAGE_BILLING = 'billing:manage';
}
```

Servicio de Autorización en el Dominio:

```
// app/Shared/Domain/Security/Authorizer.php
namespace App\Shared\Domain\Security;

use App\Shared\Domain\Enums\Permission;
use DomainException;

final class Authorizer
{
    public static function authorize(array $userPermissions, Permission $requiredPermission):
    {
        if (!in_array($requiredPermission->value, $userPermissions, true)) {
            throw new DomainException("Acceso denegado: Se requiere el permiso '{$requiredPerm
        }
    }
}
```

3. Blindaje contra Fuga de Datos (*Cross-Tenant Leakage*)

Al escribir adaptadores de persistencia en slices de aplicaciones SaaS, cada sentencia SQL debe parametrizar e incluir de forma obligatoria el `tenant_id` :

```
public function findById(string $id): ?Shift
{
    $tenantId = TenantContext::get();

    $stmt = $this->pdo->prepare("
        SELECT * FROM shifts
        WHERE id = :id AND tenant_id = :tenant_id
        LIMIT 1
    ");
    $stmt->execute([
        'id' => $id,
        'tenant_id' => $tenantId
    ]);

    $row = $stmt->fetch(PDO::FETCH_ASSOC);
    return $row ? Shift::fromState($row) : null;
}
```

Este enfoque previene que cualquier usuario pueda acceder o mutar registros de otra empresa modificando los IDs en la URL o en el cuerpo de una petición HTTP.

Capítulo 13: Proyecto Completo de Extremo a Extremo (SaaS Transaccional)

Para consolidar todos los conceptos aprendidos a lo largo de este libro, en este capítulo construiremos **"TurnoExpress Enterprise"**: un módulo SaaS transaccional de extremo a extremo para la publicación, reclamo y monitoreo en tiempo real de turnos laborales de emergencia.

1. El Caso de Uso: ClaimShift (Reclamar Turno)

Generamos el slice transaccional utilizando el CLI de HexaGenPHP:

```
php hexagen make:slice ClaimShift
```

El Comando DTO

```
// app/Slices/ClaimShift/ClaimShiftCommand.php
namespace App\Slices\ClaimShift;

final readonly class ClaimShiftCommand
{
    public function __construct(
        public string $shiftId,
        public string $workerId
    ) {}
}
```

El Manejador de Dominio (Handler)

```
// app/Slices/ClaimShift/ClaimShiftHandler.php
namespace App\Slices\ClaimShift;

use App\Slices\ShiftManagement\Ports\ShiftRepositoryPort;
use App\Shared\Domain\Ports\EventBusPort;
use App\Slices\ShiftManagement\Events\ShiftClaimedDomainEvent;
use DomainException;

final class ClaimShiftHandler
{
    public function __construct(
        private readonly ShiftRepositoryPort $repository,
        private readonly EventBusPort $eventBus
    ) {}

    public function handle(ClaimShiftCommand $command): void
    {
        $shift = $this->repository->findById($command->shiftId);

        if (!$shift) {
            throw new DomainException("El turno solicitado no existe.");
        }

        // Lógica de negocio encapsulada y protegida dentro de la entidad
        $shift->claimBy($command->workerId);

        // Persistir los cambios mediante el puerto desacoplado
        $this->repository->save($shift);

        // Publicar el evento de dominio para procesos secundarios
        $this->eventBus->publish(new ShiftClaimedDomainEvent(
            shiftId: $command->shiftId,
            workerId: $command->workerId
        ));
    }
}
```

2. El Componente Reactivo en Vivo (ShiftFeedLiveSlice)

Creamos el componente interactivo para que los trabajadores acepten el turno al instante desde su navegador:


```
php hexagen make:live ShiftFeed
```

```
// app/Slices/ShiftFeed/ShiftFeedLiveSlice.php
namespace App\Slices\ShiftFeed;

use HexaGen\Live\LiveComponent;
use App\Slices\ShiftManagement\Ports\ShiftRepositoryPort;
use App\Slices\ClaimShift\ClaimShiftHandler;
use App\Slices\ClaimShift\ClaimShiftCommand;

final class ShiftFeedLiveSlice extends LiveComponent
{
    public string $currentWorkerId = 'worker_123';
    public string $filter = 'ALL';

    public function claim(string $shiftId, ClaimShiftHandler $handler): void
    {
        $handler->handle(new ClaimShiftCommand($shiftId, $this->currentWorkerId));
    }

    public function render(ShiftRepositoryPort $repository): string
    {
        $shifts = $repository->findAvailableShifts(10);

        return $this->view('ShiftFeed/view', [
            'shifts' => $shifts
        ]);
    }
}
```

3. Plantilla Reactiva con HTMX y Tailwind CSS

```

<!-- app/Slices/ShiftFeed/view.blade.php -->
<div class="max-w-2xl mx-auto p-4 space-y-4"
    data-live-component="ShiftFeedLiveSlice"
    data-live-state="{{ $this->getEncryptedState() }}">

    <div class="flex justify-between items-center border-b pb-3">
        <h1 class="text-xl font-bold text-slate-800">Turnos Disponibles</h1>
        <span class="px-2.5 py-0.5 rounded-full text-xs font-medium bg-emerald-100 text-emerald-600">
            En vivo
        </span>
    </div>

    @forelse($shifts as $shift)
        <div class="bg-white border rounded-xl p-4 shadow-sm flex items-center justify-between">
            <div>
                <h3 class="font-bold text-slate-900">{{ $shift->title }}</h3>
                <p class="text-sm text-slate-500">{{ $shift->location }} · {{ $shift->schedule }}
                <div class="text-lg font-extrabold text-emerald-600 mt-1">{{ $shift->hourlyRate }}
            </div>

            <button hx-post="/live/ShiftFeedLiveSlice/claim?shiftId={{ $shift->id }}"
                hx-target="closest [data-live-component]"
                hx-swap="outerHTML"
                class="px-4 py-2 bg-indigo-600 hover:bg-indigo-700 text-white font-semibold">
                Aceptar Turno
            </button>
        </div>

    @empty
        <div class="text-center py-8 text-slate-400">
            No hay turnos disponibles en este momento.
        </div>

    @endforelse
</div>

```

Capítulo 14: Testing Automatizado, Dockerización, CI/CD y Rendimiento

La verdadera prueba de una arquitectura no ocurre en el entorno de desarrollo local, sino cuando el sistema se enfrenta a cargas reales, intentos de vulneración y despliegues continuos. HexaGenPHP simplifica tanto las pruebas automatizadas como el despliegue gracias a su bajo acoplamiento y su diseño sin estado (*stateless*).

1. Estrategia de Testing en Vertical Slices

Debido a que cada slice aísla sus dependencias mediante contratos (Puertos), probar las reglas de negocio no requiere levantar una base de datos ni configurar servidores web pesados.

Pruebas Unitarias de Dominio con PHPUnit

```
// tests/Unit/ClaimShiftHandlerTest.php
namespace Tests\Unit;

use PHPUnit\Framework\TestCase;
use App\Slices\ClaimShift\ClaimShiftHandler;
use App\Slices\ClaimShift\ClaimShiftCommand;
use App\Slices\ShiftManagement\Entities\Shift;
use App\Slices\ShiftManagement\Ports\ShiftRepositoryPort;
use App\Shared\Domain\Ports\EventBusPort;
use App\Shared\Domain\ValueObjects\Money;

final class ClaimShiftHandlerTest extends TestCase
{
    public function test_worker_can_claim_available_shift(): void
    {
        $shift = Shift::create('shift_1', 'Cajero Nocturno', Money::fromFloat(750.00));

        $repositoryMock = $this->createMock(ShiftRepositoryPort::class);
        $repositoryMock->expects($this->once())
            ->method('findById')
            ->with('shift_1')
            ->willReturn($shift);

        $repositoryMock->expects($this->once())
            ->method('save')
            ->with($this->callback(fn(Shift $s) => $s->toState()['status'] === 'CLAIMED'));

        $eventBusMock = $this->createMock(EventBusPort::class);
        $eventBusMock->expects($this->once())->method('publish');

        $handler = new ClaimShiftHandler($repositoryMock, $eventBusMock);
        $handler->handle(new ClaimShiftCommand(shiftId: 'shift_1', workerId: 'worker_99'));

        $this->assertEquals('CLAIMED', $shift->toState()['status']);
    }
}
```

2. Dockerfile Optimizado para Producción

```
FROM php:8.3-fpm-alpine

# Instalar dependencias del sistema y extensiones de PHP
RUN apk add --no-cache openssl-dev libpng-dev libzip-dev oniguruma-dev \
    && docker-php-ext-install pdo pdo_mysql opcache bcmath

# Copiar configuración optimizada de PHP y Opcache
COPY ./docker/php/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

WORKDIR /var/www/html

# Instalar dependencias mediante Composer
COPY composer.json composer.lock ./
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer && composer install --no-dev --optimize-autoloader --no-interaction

COPY . .

# Asignar permisos al directorio de almacenamiento
RUN chown -R www-data:www-data /var/www/html/storage

EXPOSE 9000

CMD ["php-fpm"]
```

3. Configuración de Nginx para Enrutamiento /live

```
server {  
    listen 80;  
    server_name misaas.com;  
    root /var/www/html/public;  
  
    index index.php;  
  
    location / {  
        try_files $uri $uri/ /index.php?$query_string;  
    }  
  
    location ~ /\.php$ {  
        fastcgi_pass app:9000;  
        fastcgi_index index.php;  
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;  
        include fastcgi_params;  
    }  
  
    location ~ /\.(!well-known).* {  
        deny all;  
    }  
}
```

4. Pipeline de Integración Continua (GitHub Actions)

```
# .github/workflows/ci.yml
name: CI Pipeline

on: [push, pull_request]

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          extensions: mbstring, pdo, openssl

      - name: Install Dependencies
        run: composer install --prefer-dist --no-progress

      - name: Run Unit & Integration Tests
        run: ./vendor/bin/phpunit --colors=always
```